

期末机考题目解析

Updated 2026-06-08 20:12 GMT+8

Compiled by 孙婧斯、生命科学学院 (2026 Spring)

E30646:缺失的第一个正数

<http://cs101.openjudge.cn/practice/30646>

总时间限制: 1000ms 单个测试点时间限制: 100ms 内存限制: 65536kB

描述

给你一个未排序的整数数组 `nums`，请你找出其中没有出现的最小的正整数。

输入

总共两行，第一行一个正整数 n ($1 \leq n \leq 100$)，第二行 n 个整数，用空格隔开，代表数组 `nums`。

输出

输出一个整数，代表数组 `nums` 中没有出现的最小正整数。

样例输入

```
1 | 3 1 2 0
```

样例输出

```
1 | 3
```

提示

$-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

思路：确定遍历左右边界。

右边界： n 个数，使缺失的第一个正数最大的方法就是给 1 到 n ，这时缺失的数是 $n+1$ 。因此遍历右边界是 $n+1$

左边界：要求正数，因此要从 1 遍历。从 0 遍历是可能的 WA 原因，这个样例中看不出来。

实现时用集合（`set`）存储所有数字，可以将查找降为 $O(1)$ ，总时间复杂度 $O(n)$ 。

代码：

```
1 | n=int(input())
2 | arr=set(map(int,input().split()))
3 | for i in range(1,n+2):
4 |     if i not in arr:
5 |         print(i)
6 |         break
```

E30930:猫猫水群聊

<http://cs101.openjudge.cn/practice/30930>

总时间限制: 5000ms 单个测试点时间限制: 100ms 内存限制: 1048576kB

描述

猫猫爱水群。猫猫每天都会统计群里面每个群友发言数量 x_i 。有一天猫猫突发奇想，想统计一下 x 的“h 因子”作为群聊的活跃度。

定义 S_k 为 $f_i \geq k$ 的 f_i 数量，那么 h 因子就是 $S_k \geq k$ 的最大的 k 。

猫猫要求出群聊的活跃度。即：如果有至少 k 个群友，他们每个人的发言数量都不少于 k 条，那么满足这一条件的最大的 k 值，就是该群聊的 h 因子。

举例：假设群友发言数排序后为：[10, 8, 5, 2, 1]。尝试 $k = 3$ ：有 3 个人的发言数（10, 8, 5）都大于或等于 3，满足条件。尝试 $k = 4$ ：只有 3 个人的发言数大于或等于 4，不满足“至少有 4 个人”的要求。因此，满足条件的最大 k 值为 3，群聊活跃度为 3。

输入

第一行包含一个整数 n ($1 \leq n \leq 5 \times 10^5$)，表示群友数量。接下来一行，包含 n 个整数 x_i ($0 \leq x_i \leq 10^9$)，表示第 i 个群友当天发言数量。

输出

输出一行，表示群聊的活跃度。

样例输入

```
1 | 22 262 128 210 223 62 70 104 61 80 44 40 6 63 94 42 18 1 13 0 0 0 0
```

样例输出

```
1 | 15
```

提示

共 50 个测试点，总输入数据不超过 70 MB。

思路一：排序后线性扫描 ($O(n \log n)$)

将数组降序排序。遍历时，第 i 个位置 (1-indexed) 已有 i 个数，若

$$x[i] \geq i \text{ 则至少有 } i \text{ 个数 } \geq i, \text{ 答案可更新为 } i; \text{ 一旦 } x[i] < i \text{ 即可停止。} \quad (1)$$

```
1 n = int(input())
2 x = list(map(int, input().split()))
3 x.sort(reverse=True)
4 ans = 0
5 for i in range(n):
6     if x[i] >= i + 1:
7         ans = i + 1
8     else:
9         break
10 print(ans)
```

思路二：二分查找 ($O(n \log n)$)

答案是一个有范围的数字，且该数字有约束条件，故二分查找

代码：

```

1  def check(x):
2      c=0
3      for num in arr:
4          if num>=x:
5              c+=1
6      return c>=x
7  n=int(input())
8  arr=list(map(int,input().split()))
9  #这是一个左闭右开的二分
10 lo=0
11 hi=n+1
12 while lo<hi:
13     mid=(lo+hi)//2
14     if check(mid):
15         lo=mid+1
16     else:
17         hi=mid
18 print(hi-1)

```

M30874:匹配队友

<http://cs101.openjudge.cn/practice/30874>

总时间限制: 50000ms 单个测试点时间限制: 1000ms 内存限制: 65536kB

描述

在某MMORPG游戏的地下城系统中，一个标准小队由 **1 名坦克 (T, Tank)**、**1 名治疗 (H, Healer)** 和 **3 名输出 (D, DPS)** 组成。假设其按照一种简单的方式匹配随机地下城队友：玩家按先后顺序进入匹配队列。每当队列中的剩余玩家满足上述职责要求时，系统会立即从队列中挑选出**最早进入匹配**的 5 名符合条件的玩家组成一支队伍。现给定某时刻的匹配队列，输出每位玩家所属的队伍编号。若有玩家未能成功组队，其队伍编号输出为 0。

输入

第一行一个整数 N ($1 \leq N \leq 10^5$)。第二行 N 个字符，由 T, H, D 组成，用空格隔开。

输出

一行由空格隔开的 N 个整数。

样例输入

```

1 Sample input 1:
2 10
3 D D T D H T D D H D
4 Sample input 2:
5 6
6 T H D D D T

```

样例输出

```

1 Sample output 1:
2 1 1 1 1 1 2 2 2 2 2
3 Sample output 2:
4 1 1 1 1 1 0

```

提示

queue, implementation

思路：有三种不同的职业，如果放到同一个队列里就很麻烦。因此给三种职业开三个队列与三个计数器。

代码：

```
1  from collections import deque
2
3  n = int(input())
4  role = input().split()
5  qt, qh, qd = deque(), deque(), deque()
6  res = [0] * n
7  cnt = 0
8
9  for i in range(n):
10     if role[i] == 'T':
11         qt.append(i)
12     elif role[i] == 'H':
13         qh.append(i)
14     else:
15         qd.append(i)
16     if qt and qh and len(qd) >= 3:
17         cnt += 1
18         res[qt.popleft()] = cnt
19         res[qh.popleft()] = cnt
20         for _ in range(3):
21             res[qd.popleft()] = cnt
22
23  print(*res)
```

M30680:森林局部排序遍历

<http://cs101.openjudge.cn/practice/30680>

总时间限制: 1000ms 内存限制: 65536kB

描述

给定一个森林（由一棵或多棵树组成）。对于森林中的每一棵树，遍历规则如下：**局部排序规则**：遍历到每个节点时，将该节点本身与其所有直接子节点的值放在一起，按从小到大的顺序进行遍历。**递归规则**：如果遇到的是子节点，则递归进入该子节点进行深度遍历。如果遇到的是当前节点本身，则输出该节点的值。**森林规则**：如果森林中存在多棵树，首先找到所有树的根节点，按照根节点的值从小到大的顺序依次对每棵树进行上述遍历。每个节点的值为互不相同的正整数。每个点值的范围 1~9999。

输入

第一行：节点总个数 n ($n < 1000$)。接下来的 n 行：每行代表一个节点的结构。第一个数是此节点的值，之后的数表示它的所有直接子节点的值。如果没有子节点，该行只有一个数。

输出

输出遍历结果，每行一个节点的值。

样例输入

```
1  4
2  15 2
3  2
4  8 20
5  20
```

样例输出

```
1 8
2 20
3 2
4 15
5 # 解释:
6 1. 该输入构成了两棵树 (一个森林):
7   - 第一棵树: 根为 15, 子节点为 2。
8   - 第二棵树: 根为 8, 子节点为 20。
9 2. 根节点集合为 {8, 15}。按规则, 先处理较小的根 8。
10 3. 局部排序遍历根 8
11 4. 局部排序遍历根 15`
```

思路:

1. **建图与找根**: 建立邻接表, 统计每个节点的入度。入度为 0 的节点即为根节点。
2. **排序遍历**: 对每棵树递归 DFS。在每个节点处, 将自身与所有直接子节点合并排序, 按序遍历: 遇到子节点则递归, 遇到自身则输出。

代码:

```
1 n=int(input())
2 graph={}
3 in_degree={}
4 for i in range(n):
5     nums=list(map(int,input().split()))
6     root=nums[0]
7     graph[root]=nums[1:]
8     if root not in in_degree:
9         in_degree[root]=0
10    for num in nums[1:]:
11        if num not in in_degree:
12            in_degree[num]=0
13            in_degree[num]+=1
14 ans=[]
15 def dfs(node):
16     l=graph[node][:]
17     l.append(node)
18     l.sort()
19     for num in l:
20         if num==node:
21             print(num)
22         else:
23             dfs(num)
24 roots=[]
25 for node in in_degree:
26     if in_degree[node]==0:
27         roots.append(node)
28 roots.sort()
29 for node in roots:
30     dfs(node)
```

M30947:Ask for Likes

<http://cs101.openjudge.cn/practice/30947>

总时间限制: 120000ms 单个测试点时间限制: 6000ms 内存限制: 524288kB

描述

小 L 也跟风发了一条 QQ 消息，随后群 u 们便开始在下面扣表情。已知 QQ 支持 n 种表情，编号 $1 \sim n$ 。现在小 L 发的消息获得了 $c[i]$ 个第 i 种表情。他想要让所有存在的表情数量之积为 x ，即所有非零的 $c[i]$ 之积为 x 。特别地，当不存在非零 $c[i]$ 时，认为积为 1。他想到了找人来继续扣表情，即进行若干（可以为零）次操作，每次选定一个 i 并令 $c[i]$ 加 1。他想考考你，通过这样的操作，是否可能达成他的目标。**每次询问独立，即他并不会真的找人来继续扣表情。**-----即：给出一个长度为 n 的非负整数数组 c （代表初始表情数），每次操作可以令任意 $c[i] \leftarrow c[i] + 1$ （即只可变大、不可变小）。现有 q 次独立的询问，每次给出一个目标值 x ，问能否通过若干次操作，使数组中**所有非零元素的乘积**刚好等于 x 。

输入

第一行，两个整数 n, q ($1 \leq n \leq 10^5, 1 \leq q \leq 100$)。第二行， n 个整数 $c[1], \dots, c[n]$ ($0 \leq c[i] \leq 10^9$)。接下来 q 行，每行一个整数 x ($1 \leq x \leq 10^9$)，表示一次询问。

输出

q 行，每行一个字符串 Yes 或 No，分别表示能或不能达成小 L 的目标。

样例输入

```
1 20 5
2 2 5 5 2 8 1 1 1 1 1 1 1 1 1 1 1 1 1 1
3 600
4 800
5 1000
6 1200
7 1400
```

样例输出

```
1 No
2 Yes
3 Yes
4 Yes
5 Yes
```

提示

样例的初始状态就是图中的情形，唯一的区别是这里假定 QQ 中只能使用这 20 种表情。

对于第一次询问，由于现在表情数量之积大于 600，继续扣表情只会将该值继续增大，故输出 No。

对于第二次询问，由于现在表情数量之积恰好为 800，他不需任何操作就能达成目标，故输出 Yes。

对于第三次询问，他只需将第 5 种表情（图中的“毛毛虫”）再扣 2 次即可达成目标，故输出 Yes。

对于第四次询问，他只需将第 1 种表情（图中的“牛气冲天”）或第 4 种表情（图中的“拜谢”）再扣 1 次即可达成目标，故输出 Yes。

对于第五次询问，他只需将第 2 种表情（图中的“咦”）或第 3 种表情（图中的“棒棒糖”）再扣 2 次，并将第 5 种表情再扣 2 次，即可达成目标，故输出 Yes。

约数分解，深度优先搜索 (DFS)

思路：

第一步：把问题说清楚

操作只能让 c_i 变大，不能变小。所以问题等价于：

$$\text{能否找到一组整数 } d[0], d[1], \dots, d[n-1], \text{ 满足 } d[i] \geq c[i] \text{ (每个都不小于原始值)}, \quad (2)$$

且所有非零 $d[i]$ 的乘积恰好等于 x ？

第二步：去掉没用的 0 和 1

0和1要么都可以当作1，要么可以成为剩余的任意数字。任意数字其实是不必要的，因为这个数字可以直接乘在最后一个有效数字上。因此0和1完全无用。

第三步：DFS思路

遍历表情包数量，对于当前的 $c[i]$ 与 x ，尝试 $d[i] \geq c[i]$ ，对于每个 $d[i]$

子问题是：

$$\text{能否找到一组整数 } d[0], d[1], \dots, d[i-1], \text{ 满足 } d[t] \geq c[t] \text{ (每个都不小于原始值)}, \quad (3)$$

且所有非零 $d[t]$ 的乘积恰好等于 $x/d[i]$ ？

终止条件：表情包列表遍历到末尾。

若末尾大于等于当前数字，就说明成功了，否则这个分支失败了。

伪码

```
1  dfs(i, re):          # 轮到第 i 个槽位，手里还剩 re 要被分配
2      如果 i 是最后一个槽位:
3          return re >= c[i]    # re 就是它的分配值，只要够大就行
4
5      枚举 x 的所有因数 d (满足 d >= c[i]) :
6          如果 re % d == 0:
7              如果 dfs(i+1, re // d) 返回 True:
8                  return True
9      return False
```

第四步：三种剪枝

剪枝一：大于 1 的数超过 30 个，全部 No

$$2^{30} \approx 10^9, \text{ 如果有超过30个 } \geq 2 \text{ 的槽位，它们的最小乘积就已经超过 } x \text{ 的上限 } 10^9 \text{ 了，必然无解。} \quad (4)$$

实践发现此剪枝基本没有效果，只有个位数毫秒的时间差异

剪枝二：后缀积下界（用于降序搜索）

提前计算后缀最小乘积：

$$\text{suf}[i] = c[i] \times c[i+1] \times \dots \times c[m-1]。 \quad (5)$$

进入第 i 个槽位时，后面所有槽位（包括当前）至少要消耗 $\text{suf}[i]$ 这么多乘积。如果手里的 re 已经比 $\text{suf}[i]$ 小了，那无论怎么分配都不够用，直接剪掉。

剪枝三：上界限制（用于升序搜索）

$$\text{当 } d \text{ 单调不降时，从第 } i \text{ 个槽位到最后还有 } m-i \text{ 个，每个都 } \geq d[i] \quad (6)$$

所以：

$$d[i]^{m-i} \leq \text{re} \implies d[i] \leq \text{re}^{1/(m-i)} \quad (7)$$

超过这个上界的因数完全不用尝试。槽位越多，这个上界越紧，效果越显著。

剪枝四：记忆化搜索

同样的"第 i 个槽位、手里还剩 re "这个状态可能在不同的搜索路径中多次遇到。用 `lru_cache` 把结果缓存起来，避免重复计算。

思路1 (78ms AC)

只搜索单调不降的分配方案+升序遍历+上界限制剪枝

```
1 from functools import lru_cache
2 import sys, math
3 sys.setrecursionlimit(1 << 30)
4
5 n, q = map(int, input().split())
6 c = list(map(int, input().split()))
7 c.sort()
8 # 跳过所有 0 和 1
9 start = 0
10 for i in range(n):
11     if c[i] <= 1:
12         start = i + 1
13
14 @lru_cache(1 << 20)
15 def dfs(i, x, curr):
16     # i: 当前处理的下标, x: 剩余乘积, curr: 上一个分配值 (保证单调)
17     if i == n - 1:
18         return c[i] <= x
19     # 上界: d[i] <= ceil(x^(1/(n-i))), 超出则无意义
20     upper = math.ceil(math.pow(x, 1 / (n - i))) + 1
21     for j in range(max(c[i], curr), upper):
22         if x % j == 0:
23             if dfs(i + 1, x // j, j):
24                 return True
25     return False
26
27 for _ in range(q):
28     x = int(input())
29     if start == n:
30         print("Yes")
31         continue
32     print("Yes" if dfs(start, x, 1) else "No")
```

思路2 (主流思路) (2000+msAC)

记忆化分解因数+倒序遍历+后缀积剪枝

```
1 import sys
2 from functools import lru_cache
3 sys.setrecursionlimit((1 << 30) - 1)
4
5 n, q = map(int, input().split())
6 arr = list(map(int, input().split()))
7 arr = sorted([v for v in arr if v > 1], reverse=True)
8 n = len(arr)
9
10 @lru_cache(maxsize=None)
11 def find(x):
```



```

12     factors = []
13     for i in range(1, int(x ** 0.5) + 1):
14         if x % i == 0:
15             factors.append(i)
16             if i != x // i:
17                 factors.append(x // i)
18     return tuple(sorted(factors, reverse=True))
19
20 if not arr:
21     for _ in range(q):
22         print("Yes")
23     exit()
24
25 # 后缀积 (上限 10^10 防止溢出)
26 mul = [1] * (n + 1)
27 for i in range(n - 1, -1, -1):
28     mul[i] = min(10 ** 10, mul[i + 1] * arr[i])
29
30 @lru_cache(maxsize=None)
31 def dfs(cur, i):
32     if cur < mul[i]:
33         return False
34     if i == n - 1:
35         return cur >= arr[i]
36     for f in find(cur):
37         if cur // f < mul[i + 1]:
38             continue
39         if f < arr[i]:
40             break
41         if dfs(cur // f, i + 1):
42             return True
43     return False
44
45 for _ in range(q):
46     x = int(input())
47     print("Yes" if dfs(x, 0) else "No")
48

```

T30913:猫猫逛公园

<http://cs101.openjudge.cn/practice/30913>

总时间限制: 60000ms 单个测试点时间限制: 3000ms 内存限制: 1048576kB

描述

黄色的树林里分出两条路， 可惜我不能同时去涉足， 我在那路口久久伫立， 向着一条路极目望去， 直到它消失在丛林深处。

猫猫要去公园散步。

公园里有 n 个岔路口，通过 m 条有向路径相互连接。在每条路径上都有一些美丽的风景。猫猫给每条路径上的风景都打了分，第 i 条路径上的风景值为 w_i ，猫猫经过一次就可以获得 w_i 的愉悦度。但是，猫猫对重复的风景也会厌倦，具体地，如果猫猫已经经过了 k 次某条路径，那么在第 $k+1$ 次经过时，风景值会比上一次经过时减少 k 。如果风景值被减少到 ≤ 0 ，猫猫将不再获得愉悦度。例如，若某路径上最初的风景区为 9，那么猫猫从第一次到第四次途经时依次能获得 9, 8, 6, 3 的愉悦度。从第五次及以后，猫猫将无法再从这条路径上获得任何愉悦度，但仍然可以经过该路径，也不会获得负的愉悦度。猫猫决定从第 s 个岔路口开始出发游览公园。请问猫猫最多能在公园里面获得多少愉悦度？猫猫很想让你帮它解决这个对猫猫自己来说非常困难的问题。于是猫猫想了一些办法，努力来帮助你。为了方便计算，猫猫贴心地帮你推导好了以下公式：

公式推导

设某边权值为 w ，最多走 $k+1$ 次（第 $k+1$ 次后愉悦值降至 ≤ 0 ）。条件为：

$$1 + 2 + \cdots + k < w_i \implies \frac{k(k+1)}{2} < w_i \quad (8)$$

即

$$k^2 + k - 2w_i < 0 \quad (9)$$

解得：

$$k < \frac{-1 + \sqrt{1 + 8w_i}}{2} \quad (10)$$

k 取满足此条件的最大整数，因此最多走 $t = k + 1$ 次，其中：

$$t = \left\lceil \frac{-1 + \sqrt{1 + 8w_i}}{2} \right\rceil \quad (11)$$

$$\text{总愉悦值为 (第 } j \text{ 次愉悦值} = w_i - \sum_{k=1}^{j-1} k = w_i - \frac{(j-1)j}{2} \text{) :} \quad (12)$$

$$\sum_{j=1}^t \left(w_i - \frac{(j-1)j}{2} \right) = w_i t - \sum_{n=0}^{t-1} \frac{n(n+1)}{2} = w_i t - \frac{1}{2} \cdot \frac{(t-1)t(2t-1)}{6} - \frac{1}{2} \cdot \frac{t(t-1)}{2} \quad (13)$$

$$\text{利用 } \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}, \sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4} \text{ 整理, 最终得:} \quad (14)$$

$$\text{joy}(w) = w_i t - \frac{(t-1)t(t+1)}{6} \quad (15)$$

$$\text{其中 } t = \left\lceil \frac{-1 + \sqrt{1 + 8w_i}}{2} \right\rceil. \quad (16)$$

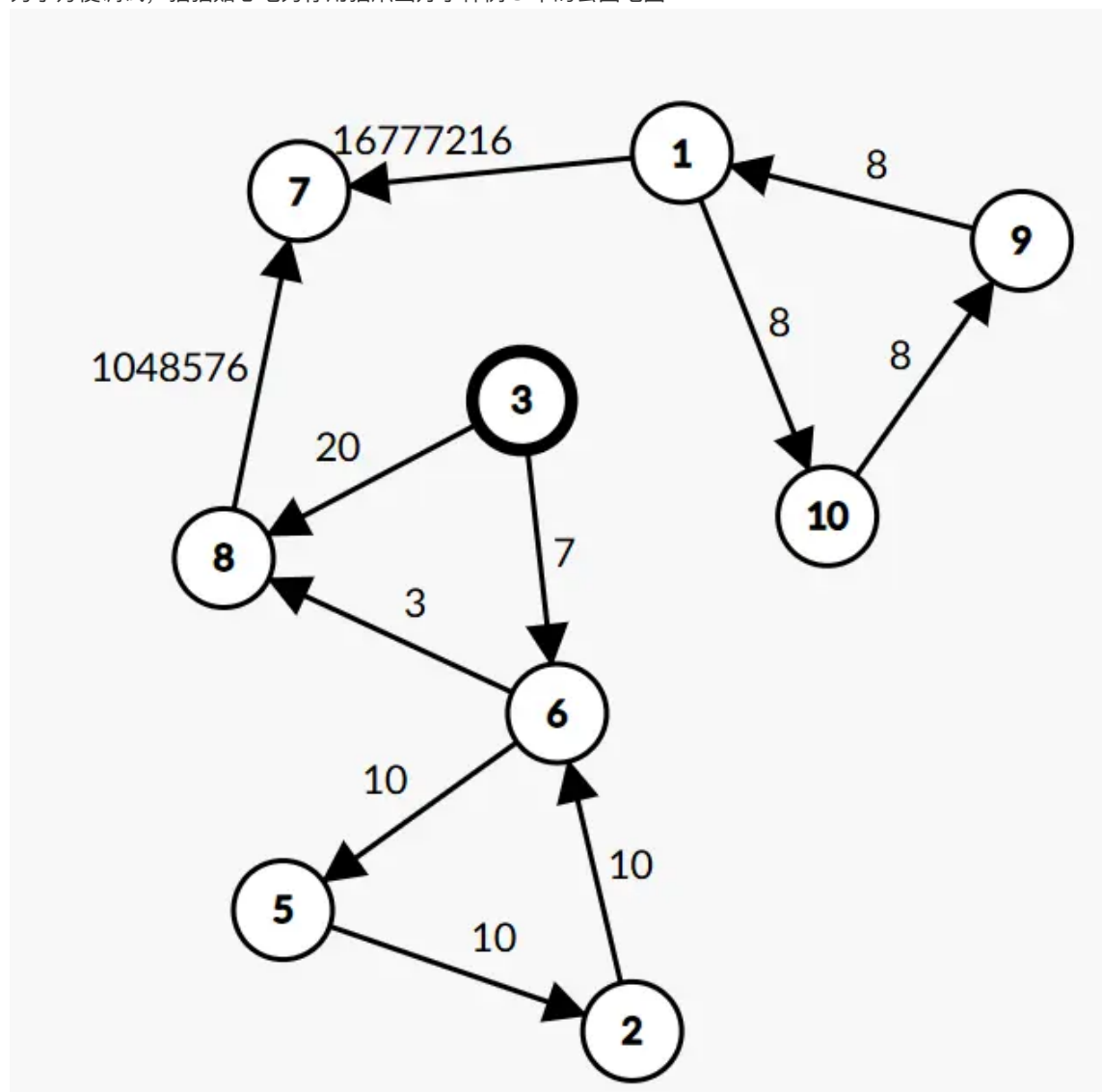
在 $K = \left\lceil \frac{\sqrt{8w_i+1}-1}{2} \right\rceil + 1$ 次游览及之后风景值会被减少到 ≤ 0 ，猫猫将不再获得愉悦度。

令： $T = K - 1$ 表示该路径最多能产生正愉悦度的游览次数。

如果猫猫选择**完全榨干**第 i 条路径（即游览次数 $\geq T$ ），能获得的最大总愉悦度：

$$W_{\text{total}} = T \cdot w_i - \frac{(T-1)T(T+1)}{6}$$

为了方便调试，猫猫贴心地为你用猫爪画好了样例 3 中的公园地图：



输入

第一行包含两个整数 n 和 m ($1 \leq n \leq 10^5$, $0 \leq m \leq 2 \cdot 10^5$)，分别表示公园里的岔路口数量和有向路径数量。接下来的 m 行，每行包含三个整数 x_i 、 y_i 和 w_i ($1 \leq x_i, y_i \leq n$, $0 \leq w_i \leq 10^8$)，表示一条从岔路口 x_i 到岔路口 y_i 的有向路径，路径上最初的风景值为 w_i 。允许从某个岔路口到自身的路径，也允许两个岔路口之间存在多条路径。最后一行包含一个整数 s ($1 \leq s \leq n$)，表示猫猫的起始位置。

输出

输出一个整数，表示猫猫在游览中最多能够获得的愉悦度。

样例输入

```

1 sample1 input:
2 2 2
3 1 2 4
4 2 1 4
5 1
6 sample2 input:
7 3 3
8 1 2 4
9 2 3 3
10 1 3 8
11 1
12 sample3 input:
13 10 11
14 1 10 8
15 10 9 8
16 9 1 8
17 1 7 16777216
18 8 7 1048576
19 3 8 20
20 3 6 7
21 6 5 10
22 5 2 10
23 2 6 10
24 6 8 3
25 3

```

样例输出

```

1 sample1 output:
2 16
3 sample2 output:
4 8
5 sample3 output:
6 1048676

```

提示

Shortest Path, SCC, Topological Order, DP（最短路径，强连通，拓扑排序，动态规划）共 100 个测试点，单个测试点输入不超过 5MB，总输入不超过 200MB。

思路：

第一步：单边最大愉悦值计算

把公式翻译成代码（注意浮点精度，建议用 `ceil`）：

```

1 import math
2 def cal(w):
3     t = math.ceil((math.sqrt(8 * w + 1) - 1) / 2)
4     return t * w - (t - 1) * t * (t + 1) // 6

```

第二步：SCC 缩点

在一个强连通分量（SCC）内部，猫猫可以任意绕行，直到所有边的愉悦值都被榨干为 0。因此 SCC 内部所有边的 `cal(w)` 之和就是该 SCC 的“点权”。SCC 之间的有向边只能走一次，记录最大边权。

用 Tarjan 算法求所有 SCC，自然得到逆拓扑序，缩点后构建 DAG。

第三步：DAG 上 DP 求最长路

从起点 s 所在的 SCC 出发，在 DAG 上做 DP。

初始化： $dp[s'] = sccw[s']$ (s' 为 s 所在 SCC 的编号)，其余节点 $dp = -\infty$ (或 -1)。 (17)

•

转移：按拓扑序遍历， $dp[v] = \max(dp[v], dp[u] + w_{uv} + sccw[v])$ 。 (18)

•

注意：只有 $dp[u] \neq -1$ (即 u 可从起点到达) 才进行转移。 (19)

•

易错点

WA：这道题中节点自环和重边都算作有效的不同边，如果代码忽略这些就会WA。

MLE：这道题数据量很大，因此如果在建邻接表和记录边权的时候不节省就会MLE。

例如：建了一个只保存节点的邻接表和一个保存边权的字典，会超空间。把边权合并到邻接表中就能通过了。

代码 (858ms)

```
1 import sys
2 import math
3 sys.setrecursionlimit(10**6)
4 def tarjan_scc(graph):
5     n=len(graph)
6     dfn=[-1]*n
7     low=[-1]*n
8     stack=[]
9     on_stack=[False]*n
10    sccs=[]
11    timer=0
12    def dfs(u):
13        nonlocal timer
14        dfn[u]=low[u]=timer
15        timer+=1
16        stack.append(u)
17        on_stack[u]=True
18        for v,w in graph[u]:
19            if dfn[v]==-1:
20                dfs(v)
21                low[u]=min(low[u],low[v])
22            elif on_stack[v]:
23                low[u]=min(low[u],dfn[v])
24        if low[u]==dfn[u]:
25            component=[]
26            while True:
27                node=stack.pop()
28                on_stack[node]=False
29                component.append(node)
30                if node==u:
31                    break
32            sccs.append(component)
33    for i in range(n):
```

```

34         if dfn[i]==-1:
35             dfs(i)
36     return sccs
37 def data():
38     for line in sys.stdin.buffer:
39         for token in line.split():
40             yield int(token)
41 def cal(w):
42     t=math.ceil(((8*w+1)**0.5-1)/2)
43     wt=t*w-((t+1)*t*(t-1))/6
44     return wt
45 def main():
46     it=data()
47     n=next(it)
48     m=next(it)
49     graph=[[] for _ in range(n)]
50     for _ in range(m):
51         u=next(it)-1
52         v=next(it)-1
53         w=next(it)
54         graph[u].append((v,w))
55     sccs=tarjan_scc(graph)
56     l=len(sccs)
57     scc_id=[0]*n
58     scc_w=[0]*l
59     for i,comp in enumerate(sccs):
60         for node in comp:
61             scc_id[node]=i
62     #缩点与权值计算
63     DAG=[dict() for _ in range(l)]
64     for u in range(n):
65         for v,w in graph[u]:
66             su,sv=scc_id[u],scc_id[v]
67             if su!=sv:
68                 if sv not in DAG[su]:
69                     DAG[su][sv]=0
70                     DAG[su][sv]=max(DAG[su][sv],w)
71             else:
72                 scc_w[su]+=cal(w)
73     #dp
74     dp=[-1]*l
75     start=next(it)-1
76     s=scc_id[start]
77     dp[s]=scc_w[s]
78     for u in range(l-1,-1,-1):
79         if dp[u]!=-1:
80             for v,w in DAG[u].items():
81                 dp[v]=max(dp[v],dp[u]+w+scc_w[v])
82     print(max(dp))
83 if __name__=='__main__':
84     main()

```

U30919:猫猫去旅行

<http://cs101.openjudge.cn/practice/30919>

总时间限制: 60000ms 单个测试点时间限制: 3000ms 内存限制: 1048576kB

描述

猫猫要去旅行。在一条长为 L 的街道上有 n 处景点，第 i 处位于 x_i 。猫猫的旅行计划一共有 n 天，第 i 天要从住处去第 i 处景点。猫猫准备在街道上选择两个地方 s_1, s_2 住，并且在第 t 天晚上从 s_1 搬去 s_2 （当然猫猫也可以不搬）。猫猫要使得自己每天往返景点

的距离之和 $2 \times \left(\sum_{i=1}^t |x_i - s_1| + \sum_{i=t+1}^n |x_i - s_2| \right)$

最短。问这个距离是多少。

输入

第一行包含一个整数 n ($1 \leq n \leq 5 \times 10^5$)，表示房屋数目。第二行包含 n 个整数 x_i ($1 \leq x_i \leq 10^9$)，两两互不相同，表示每天要去的位置。

输出

输出一个整数，表示最短距离。

样例输入

```
1 | 9
2 | 3 4 1 9 2 12 6 5 7
```

样例输出

```
1 | 36
```

提示

1、Heap, Math（堆，数学） 2、共 50 个测试点，单个测试点输入不超过 5MB，总输入不超过 120MB。

思路：

关键结论

$$\sum_{i=1}^t |x_i - s| \text{ 在 } s \text{ 取 } x_1, \dots, x_t \text{ 的中位数时最小，最小值为：} \quad (20)$$

$$\text{cost}(t) = \sum_{i=1}^t |x_i - \text{median}(x_1, \dots, x_t)| \quad (21)$$

因此：

$$\text{枚举搬家时间 } t \in 0, 1, \dots, n. \quad (22)$$

1.

$$\text{预计算数组前缀的最优代价 } \text{pre}[t] \text{（前 } t \text{ 个元素以中位数为住处的总偏差）。} \quad (23)$$

2.

$$\text{预计算数组后缀的最优代价 } \text{suf}[t] \text{（后 } n - t \text{ 个元素的总偏差）。} \quad (24)$$

3.

$$\text{答案为 } 2 \times \min_{t=0}^n (\text{pre}[t] + \text{suf}[t])。 \quad (25)$$

4.

对顶堆动态维护中位数及总偏差

用两个堆动态维护：

- `left`：大顶堆，存较小的一半（Python 中用负值模拟）
- `right`：小顶堆，存较大的一半

维护 `len(left) == len(right)` 或 `len(left) == len(right) + 1`，

此时中位数为 `left` 的堆顶（`-left[0]`）。

同时用 `s_left`、`s_right` 分别维护两个堆的元素之和，则当前总偏差为：

$$\text{cost} = s_{\text{right}} - s_{\text{left}} - \text{median} \times (\text{lenright} - \text{lenleft}) \quad (26)$$

证明：若 `len(left) == len(right)`，中位数取哪个都行，代价就是

$$s_{\text{right}} - s_{\text{left}}; \quad (27)$$

若 `len(left) == len(right) + 1`，代价为

$$s_{\text{right}} - s_{\text{left}} - \text{median} \quad (28)$$

直接按

$$s_{\text{right}} - s_{\text{left}} - \text{midian} \times (\text{lenleft} - \text{lenright}) \quad (29)$$

统一表达即可。

代码

```

1 import heapq
2 n=int(input())
3 points=list(map(int,input().split()))
4 def mid(arr):
5     l=[0]*(n+1)
6     left=[]
7     right=[]
8     re=arr[0]
9     heapq.heappush(left,-re)
10    sum_left=re
11    sum_right=0
12    for i in range(1,n):
13        re=arr[i]
14        if re<=-left[0]:
15            heapq.heappush(left,-re)
16            sum_left+=re
17        else:
18            heapq.heappush(right,re)
19            sum_right+=re
20        # 保持平衡: len(left) == len(right) 或 len(left) == len(right)+1
21        if len(left)>len(right)+1:
22            m=-heapq.heappop(left)
23            sum_left-=m
24            heapq.heappush(right,m)

```



```
25         sum_right+=m
26         elif len(right)>len(left):
27             m=heapq.heappop(right)
28             sum_right-=m
29             heapq.heappush(left,-m)
30             sum_left+=m
31         l[i+1]=sum_right-sum_left-left[0]*(len(left)-len(right))
32     return l
33 pre_l=mid(points)
34 pre_r=mid(points[::-1]) #反转复用
35 ans=float('inf')
36 for t in range(n+1):
37     s=pre_l[t]+pre_r[n-t]
38     ans=min(ans,s)
39 print(2*ans)
```